GPU-Accelerated Stochastic Cellular Automaton Method for 2D Austenite-Ferrite phase transformation kinetics in Steel

Jarrod Angove — 1576192 — MATE 664

1. Introduction and Motivation

The microstructural evolution of steel during cooling dictates its final macroscopic mechanical properties, such as yield strength and toughness. The primary area of interest for this study is the phase transformation from face-centered cubic (FCC) austenite to body-centered cubic (BCC) ferrite. This is a heterogeneous, kinetically driven process governed by thermodynamic driving forces, interfacial energies, and boundary mobility. Understanding and predicting the morphology and final average grain size is highly impactful, as these targets directly dictate the material's suitability for a wide range of applications.

While the equilibrium state of this system can be mapped using conventional CALPHAD methods, the actual industrial processing of steel often relies on rapid temperature changes well beyond equilibrium conditions. Considerable effort has been made to model the kinetics of this system on a macroscopic level (i.e. JMAK kinetics [1], [2], LD-type models [3], [4], or Kirkaldy-Li type models [5], [6]). However, these models are limited in their ability to account for instantaneous geometric state (microstructure/morphology).

Others have tried to capture this geometric state through the application of 2D or even 3D spatially resolved models, such as phase field or cellular automata methods [7], [8], [9], [10]. While these models provide useful insights into the mechanism of different transformations, they are often too slow for real-world (predictive) applications.

To address performance bottlenecks, this work explores a semi-quantitative stochastic cellular automaton method for predicting the evolution of austenite-ferrite phase transformations. The proposed algorithm is implemented in a GPU-compatible fashion, allowing faster-than-real-time predictions for sufficiently slow transformations, even on consumer-grade hardware.

2. Thermodynamics

The fundamental governing theory for this phase transformation is the minimization of the total free energy. The domain is discretized into a uniform 2D Cartesian grid of voxels, each characterized by a physical length l, a phase state $p \in \{0, 1\}$ (where 0 is austenite and 1 is ferrite), and a crystallographic orientation angle θ.

The total free energy of any local voxel is the sum of its volumetric free energy and its interfacial energy contributions from its four nearest neighbors:

$$E_{\text{total}} = G_v(T, p)l^3 + \sum_{i=1}^{4} \gamma_{\text{inter}}(\Delta\theta_i, p, p_i)l^2 \quad (1)$$

The volumetric Gibbs free energy ($G_v(T, p)$) for both phases is taken directly from Thermo-Calc 2025b [11] calculations for a 1.02-Mn, 0.05-Mo, 0.05-C steel. To avoid interfacing directly with Thermo-Calc, the free energy was calculated for a range of temperatures, then second-order polynomials were fit for use as surrogate models. The resulting free energies, in units of eV/µm^3, are:

$$\begin{aligned} G_{\text{ferrite}} &= -1.52 \cdot 10^4 T^2 - 3.33 \cdot 10^7 T \\ &\quad -4.26 \cdot 10^9 \\ G_{\text{austenite}} &= -8.31 \cdot 10^3 T^2 - 4.83 \cdot 10^7 T \\ &\quad +3.12 \cdot 10^9 \end{aligned} \quad (2)$$

Where T is in units of °C. It should be noted that, in reality, the free energy of both phases is highly sensitive to carbon content. To keep the scope of this work reasonable, carbon diffusion is being neglected. However, it is possible to utilize free energy functions that depend on both temperature and carbon concentration, which would produce a more accurate model.

This method also neglects the presence of dislocations in the material. The interplay between free energy and dislocation density has been explored in the literature [8] in the context of phase field modeling, but it is beyond the scope of this work. It is important to note, however, that real industrial steel processing relies heavily on the ability to introduce plastic deformation in a controlled manner. Future work will investigate this in greater detail.

In contrast to free energy, interface energy is very difficult to measure and therefore predict [1]. This work will not begin to address this issue – it will be assumed that the austenite-ferrite interface energy is on the order of 1 J/m^2, in line with the values suggested by [7],

[12]. From here, an orientation-dependence is roughly approximated by the following relation:

$$\gamma_{\text{inter}} = \begin{cases} \gamma_{\gamma/\alpha}(1 + \beta \sin(\Delta\theta)) & \text{if inter.} = \gamma/\alpha \\ \gamma_{\alpha/\alpha} \sin(\Delta\theta) & \text{if inter.} = \alpha/\alpha \\ \gamma_{\gamma/\gamma} \sin(\Delta\theta) & \text{if inter.} = \gamma/\gamma \end{cases} \quad (3)$$

Where β is an empirical factor that determines the influence of coherence on total interface energy. $\Delta\theta$ is the difference in orientation angle between the adjacent voxels. This produces a rough, qualitative dependence on interface coherence; Future work will include an implementation of an interfacial energy model that is derived from first principles.

3. Kinetics

The temporal evolution of the microstructure is governed by local thermodynamic driving forces acting on discrete interfaces. Phase transformations and boundary migrations are permitted only when they result in a reduction of the system's total free energy ($\Delta E < 0$). For any energetically favorable candidate state, the interface velocity, v, is calculated as the product of the thermodynamic driving force (ΔE) and the local interface mobility, M:

$$v = M\Delta E \quad (4)$$

$$M = M_0 \exp\left(-\frac{Q}{RT}\right) \quad (5)$$

While the pre-exponential factor M_0 and the activation energy Q can be treated as bulk constants, real physical systems exhibit significant kinetic anisotropy at structural defects.

3.1. Kinetic Anisotropy on High-Angle Grain Boundaries

At the mesoscale, standard thermodynamic driving forces scale volumetrically (l^3), overpowering the interfacial energy penalties (l^2) that naturally cause new phases to wet the grain boundaries of the parent phase. Uncorrected, this imbalance artificially forces isotropic, spherical growth.

To capture physically accurate allotriomorphic morphologies, a spatially dependent mobility model is implemented. Grain boundaries—specifically high-angle boundaries—are open, defective structures that provide high-speed atomic diffusion pathways. In the simulation, if a voxel being consumed sits on a boundary between two grains of its *own* parent phase with a misorientation exceeding a critical high-angle threshold (θ_{HA}), its mobility is scaled by a kinetic multiplier, M_{GB}:

$$M_{\text{local}} = M \cdot M_{\text{GB}} \quad (6)$$

This approach ensures that advancing fronts grow preferentailly along existing high-angle grain boundaries before thickening into the bulk grains, mimicking both heating (austenite growth) and cooling (ferrite growth) kinetics based purely on local crystallographic topology.

3.2. Linear Capture Probability and Transition Selection

To map the physical interface velocity to the discrete mesoscale grid, the model utilizes a direct geometric interpretation. The probability of a voxel transitioning to a neighboring state is defined as the fraction of the voxel's characteristic length, l, that the advancing interface traverses during the time step, Δt:

$$P_{\text{capture}} = \frac{v\Delta t}{l} \quad (7)$$

Because a single voxel may be bounded by multiple favorable neighbor states simultaneously, the competing capture probabilities stack physically. The cumulative probability of transformation is calculated, and a uniform random variable is used to select the successful state. This is equivalent to constructing and sampling from a categorical distribution over the neighbouring states.

3.3. Adaptive Time-Stepping and Stability

Because the linear capture probability must never exceed 1.0 (which would physically represent an interface jumping more than an entire voxel in a single step), a fixed time step cannot guarantee mathematical stability given the highly variable local mobilities and temperature profiles.

Instead, the simulation employs a strict Courant–Friedrichs–Lewy (CFL) adaptive time-stepping architecture. At the start of every macroscopic iteration, the peak theoretical velocity across the entire grid, v_{max}, is evaluated using a hardware-accelerated parallel reduction. The safe micro-step size is dynamically calculated to ensure the maximum transition probability never exceeds an upper limit (P_{max}):

$$\Delta t_{\text{safe}} = \frac{P_{\text{max}} \cdot l}{v_{\text{max}}} \quad (8)$$

By calculating the shortest reachable time scale at every macro-step, each iteration can be split into a collection of micro-steps based on the value of Δt_{safe}. This prevents large performance hits at high temperatures, where the kinetics are fast, but the absence of growing phases makes the maximum interface velocity relatively slow.

3.4. Nucleation Sub-Grid Scaling

An important limitation of calculating transition probabilities on the macro-scale is the inability to naturally handle nucleation events. This is because the $\sim 1\mu\text{m}$ pixels are significantly larger than the critical nucleation radius.

As an example, the volumetric driving force, based on the Gibbs free energies calculated in this work, is roughly $-4.1 \cdot 10^{-8}\text{J/m}^3$ at 700°C. Assuming an interface energy of $1\ \text{J/m}^2$, classical nucleation theory would suggest a critical radius of about 48 nm. Even this is an overly-optimistic estimate, as it does not account for nuclei that form at defects or grain boundaries, which would have significantly smaller critical radii.

To circumvent this issue, the main Cellular Automaton update loop is split into nucleation and growth passes. While the growth phase evaluates the linear capture probability strictly among nearest neighbors, the nucleation phase randomly selects a state from all possible states (any orientation and any phase) for every pixel. The probability of this transition is calcualted via a combination of the linear capture probability and a metropolis criterion;

$$\Gamma_{i\to j} = \begin{cases} K_{\text{nuc}} P_{\text{capture}} & \text{if } \Delta E < 0 \\ K_{\text{nuc}} P_{\text{capture}} \exp\left(-\frac{\Delta E}{k_B T}\right) & \text{if } \Delta E \geq 0 \end{cases} \tag{9}$$
$$P_{\text{nuc}} = 1 - \exp\left(\Gamma_{i\to j}\Delta t\right)$$

The updated state is then accepted with probability P_{nuc}. For the purpose of simplicity, ΔE is calculated in the same way as it is for the growth model. In reality, the activation energy for nucleation is unlikely to be equal to the activation energy for growth. While highly unphysical, this deficiancy can be limited by tuning K_{nuc} until the nucleation kinetics present the desired behavour.

The primary limitaiton of this approach is that K_{nuc} effectively acts as a nucleation 'fudge factor', that is not based on any physical theory. As it is not tied to the lengthscale of the simulation, different simulation lengthscales (i.e. different values of l) can results in dramatically different nucleation behaviour. Future work will investigate more rigorous approaches to nucleation kinetics.

4. Computational Details

The simulation utilizes the Julia programming language, specifically the `AMDGPU.jl` package [13], to perform highly parallelized calculations. All packages used in the module are fully open-source.

The entire module for this simulation is less than 900 lines of code – a large portion of which is plotting and initialization utilities. The full module can be seen in the *Appendix*.

4.1. Red-Black Parallelization

A standard Cellular Automata approach operating simultaneously on all grid points suffers from race conditions. To resolve this, a Red-Black checkerboard algorithm is employed. The grid is divided into two parities. During a micro-step, the GPU exclusively updates "Red" pixels while holding "Black" pixels constant, then reverses. This preserves thermodynamic conservation while allowing massive parallelization.

4.2. Initial Conditions

To initialize the simulations, a nearest neighbour (sometimes referred to as vornoi tesselation) algorithm was used to generate an equiaxed grain structure. The number of seeds for the initial generation was chosen such that the microstructure had grains on the order of 20 to 50 μm, which is common for some hot-rolled steels.

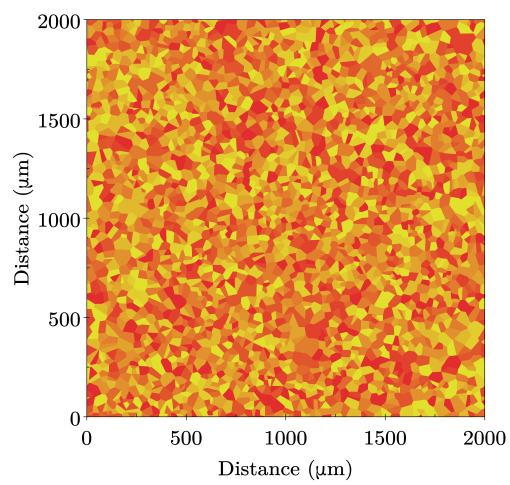

Figure 1: Initial microstructure used for the benchmark simulation. Each pixel is 0.5 μm × 0.5μm; 4000 × 4000 pixels; Total domain is 2 mm × 2 mm.

It should be noted that, in real hot-rolling, austenite grains are rarely equiaxed before ferrite begins to nucleate. The rolling process usually results in flattened grains with high dislocation densities and more grain boundaries. For simplicity, this has been neglected here.

5. Kinetic time scale

The Δt_{safe} variable that results from the CFL condition in the adaptive time-stepping section provides a simple measure of the timescale of the reaction. It characterizes the smallest possible time span in which something 'interesting' may happen. As shown in Equation 8, the simulation time scale is directly proportional to the length scale, and inversely proportional to the velocity. Some intuition about this value can be gained by substituting Equation 4 into Equation 8:

$$\Delta t_{\text{safe}} = \frac{l P_{\max}}{M_0 \Delta E} \exp\left(\frac{Q}{RT}\right) \tag{10}$$

This shows that the time scale of the system is inversely proportional to the mobility; increasing the activation energy results in an exponential increase in the time scale, while increasing the temperature exponentially decreases it. This expontial dependence on temperature is critical for non-isothermal transformations, as it prevents transformations that may be energetically favourable if they do not overlap with the temperature at which their interfaces are mobile. By differentiating the exponential term, we can see a more explicit dependence on the cooling rate:

$$\frac{\mathrm{d}}{\mathrm{d}t} \exp\left(\frac{Q}{RT(t)}\right) = -\frac{Q\dot{T}}{T^2} \exp\left(\frac{Q}{RT(t)}\right) \tag{11}$$

This shows that the timescale decreases linearly with an increase in cooling rate. In this sense, there are two competing time scales: the interface mobility time scale, and the temperature time scale, which is dictated by the heating/cooling rates.

6. Results

To benchmark the methodology, a simulation was run with the initial conditions shown in Figure 1 ($4000 \times 4000 = 1.6 \cdot 10^7$ pixels). The thermal history for this test is shown in Figure 2. A periodic temperature profile, with a maximum cooling/heating rate of 1.1 °C/s was selected to demonstrate the hysteresis of the austenite-ferrite transformation. The parameters for the simulation are shown in Table 1.

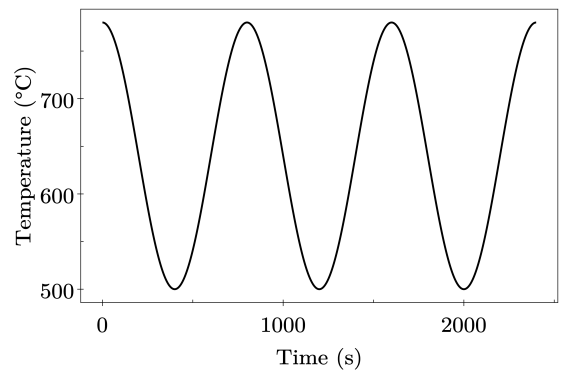

Figure 2: Thermal history for the benchmark simulation.

Table 1: Parameters used for benchmark simulation.

Parameter	Value
β	0.9
$\gamma_{\alpha/\alpha}$	$4.5 \cdot 10^6$ eV/μm^2
$\gamma_{\gamma/\gamma}$	$4.1 \cdot 10^6$ eV/μm^2
$\gamma_{\gamma/\alpha}$	$7.0 \cdot 10^6$ eV/μm^2
Q	$1.7 \cdot 10^5$J/mol
M_0	$1.8 \cdot 10^{-4}$m^4/Js
K_{nuc}	$6 \cdot 10^{-6}$
l	0.5μm
θ_{HA}	20°
M_{GB}	6

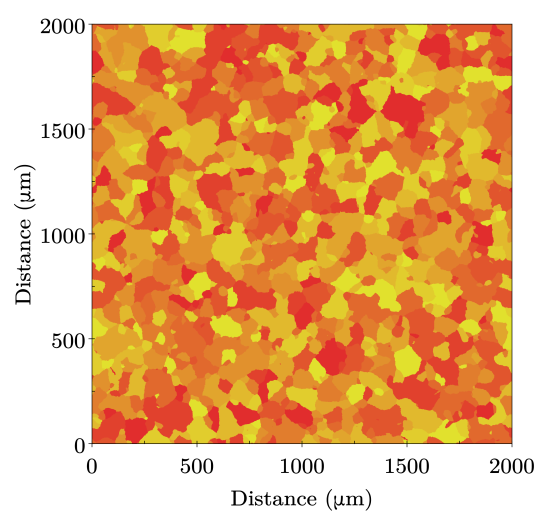

Figure 3: Final state of the benchmark simulation.

A video of the evolution of the microstructure can be viewed *here*. The final microstructure is shown in Figure 3.

6.1. Microstructure statistics

The kinetics of phase transformations are often summarized by the evolution of microstructure statistics. Grain size (Figure 4) and area fraction transformed (Figure 5) are calculated for each macro-step.

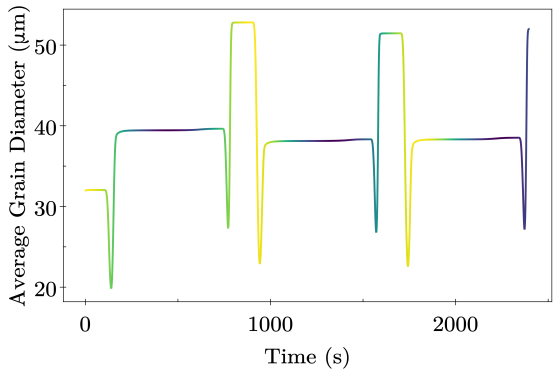

Figure 4: Average grain size as a function of time.

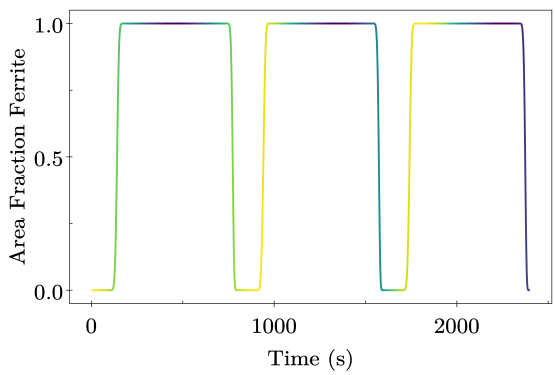

Figure 5: Area fraction of ferrite as a function of time.

The ability to calculate these statistics opens this work to the possibility of model tuning. While it is not feasible to measure the complete morphology of a steel microstructure in-situ, there are multiple experimental methods that can approximate the total volume fraction of individual phases [14], [15], [16].

6.2. Performance

The simulated time of the thermal history was 2400 seconds (40 minutes), with 1 second time steps, for a total of 2400 macro-steps. Due to the adaptive time stepping, the total number of real iterations (i.e. how many times the state was updated) was roughly 12,400.

Despite the fine grid and long simulation time, the average compute time across 15 replicates was only 158.9 seconds, with a standard deviation of 2.2 seconds. This results in an iteration frequnecy of approximately 78 iterations/s, or 0.013 s/iteration. Reducing the gride size to 1000 by 1000 yields similar microstructural statistics with simulations that take ~7 seconds.

This benchmark simulation was run on a system with an AMD Ryzen 9 7900X (12 core) processor, and an AMD Radeon RX 6800 GPU. As almost all of the simulation runs on custom GPU kernels, it is fully bound by the performance of the GPU. As these GPU kernels are highly memory-bound, a top-end accelerator like the AMD Instinct MI300 or NVIDIA H100, may be capable of doing a similar simulation in as little as 16 to 34 seconds (owing to their 5 - 10× memory bandwidth).

In the context of model tuning, this level of performance opens the door to Bayesian optimization in the span of days or even hours. As a general rule of thumb, [17] argues that Gaussian process models require roughly $10D$ observations, where D is the dimensionality of the loss function. If all 7 of the model parameters were tuned, this would require roughly 70 realizations of the same simulation. For a 4000 by 4000 grid, this would take roughly 3 hours. For a 1000 x 1000 grid, this may be possible in under 10 minutes.

7. Conclusion

The proposed GPU accelerated stochastic CA model is capable of producing semi-quantitative microstructure predictions with faster-than-real-time performance. While this work has not attempted to produce results that align with experimental observations, the computational performance of the algorithm may act as a proof of concept for more physically rigorous models.

Future work will investigate the viability of tuning model parameters to measurements collected on real steel samples.

Bibliography

[1] M. Militzer, C. Hutchinson, H. Zurob, and G. Miyamoto, "Modelling of the Diffusional Austenite-Ferrite Transformation," *International Materials Reviews*, vol. 68, no. 7, pp. 725–754, Oct. 2023, doi: *10.1080/09506608.2022.2126257*.

[2] S. J. Song, W. K. Che, J. B. Zhang, L. K. Huang, S. Y. Duan, and F. Liu, "Kinetics and Microstructural Modeling of Isothermal Austenite-to-Ferrite Transformation in Fe-C-Mn-Si Steels," *Journal of Materials Science & Technology*, vol.

35, no. 8, pp. 1753–1766, Aug. 2019, doi: *10.1016/j.jmst.2019.04.010*.

[3] J. B. Leblond and J. Devaux, "A New Kinetic Model for Anisothermal Metallurgical Transformations in Steels Including Effect of Austenite Grain Size," *Acta Metallurgica*, vol. 32, no. 1, pp. 137–146, Jan. 1984, doi: *10.1016/0001-6160(84)90211-6*.

[4] I. Milenin, R. Kuziak, Ł. Rauch, and M. Pietrzyk, "Model of Phase Transformations in Steels Subject to Heating-Cooling Thermal Cycles in Continuous Annealing Line," *Canadian Metallurgical Quarterly*, vol. 58, no. 3, pp. 367–377, July 2019, doi: *10.1080/00084433.2019.1590041*.

[5] J. S. Kirkaldy, "Prediction of Microstructure and Hardenability in Low Alloy Steels," *Proceedings of the International Conference on Phase Transformation in Ferrous Alloys, 1983*, 1983, Accessed: Sept. 04, 2025. [Online]. Available: *https://cir.nii.ac.jp/crid/1570572701698379776*

[6] M. V. Li, D. V. Niebuhr, L. L. Meekisho, and D. G. Atteridge, "A Computational Model for the Prediction of Steel Hardenability," *Metallurgical and Materials Transactions B*, vol. 29, no. 3, pp. 661–672, June 1998, doi: *10.1007/s11663-998-0101-3*.

[7] M. Militzer, "Computer Simulation of Microstructure Evolution in Low Carbon Sheet Steels," *ISIJ International*, vol. 47, no. 1, pp. 1–15, 2007, doi: *10.2355/isijinternational.47.1*.

[8] Y.-G. Cho *et al.*, "Analysis of Transformation Plasticity in Steel Using a Finite Element Method Coupled with a Phase Field Model," *PLOS ONE*, vol. 7, no. 4, p. e35987, Apr. 2012, doi: *10.1371/journal.pone.0035987*.

[9] M. G. Mecozzi, C. Bos, and J. Sietsma, "3D Cellular Automata Modelling of Solid–State Transformations Relevant in Low–Alloy Steel Production," *Solid State Phenomena*, pp. 1140–1145, June 2011, doi: *10.4028/www.scientific.net/SSP.172-174.1140*.

[10] M. G. Mecozzi, C. Bos, and J. Sietsma, "Microstructure Modelling of Solid-State Transformations in Low-Alloy Steel Production \textsuperscript{}," *Materials Science Forum*, pp. 2782–2787, Jan. 2012, doi: *10.4028/www.scientific.net/MSF.706-709.2782*.

[11] J.-O. Andersson, T. Helander, L. Höglund, P. Shi, and B. Sundman, "Thermo-Calc & DICTRA, Computational Tools for Materials Science," *Calphad*, vol. 26, no. 2, pp. 273–312, June 2002, doi: *10.1016/S0364-5916(02)00037-8*.

[12] J. J. Wits, T. A. Kop, Y. van Leeuwen, J. Seitsma, and S. van Der Zwaag, "A Study on the Austenite-to-Ferrite Phase Transformation in Binary Substitutional Iron Alloys," *Materials Science and Engineering: A*, vol. 283, no. 1, pp. 234–241, May 2000, doi: *10.1016/S0921-5093(00)00735-8*.

[13] J. Samaroo *et al.*, "JuliaGPU/AMDGPU.Jl: V2.3.0." Accessed: Apr. 03, 2026. [Online]. Available: *https://zenodo.org/records/19035609*

[14] S. Lin, U. Borggren, A. Stark, A. Borgenstam, W. Mu, and P. Hedström, "In-Situ High-Energy X-ray Diffraction Study of Austenite Decomposition During Rapid Cooling and Isothermal Holding in Two HSLA Steels," *Metallurgical and Materials Transactions A*, vol. 52, no. 5, pp. 1812–1825, May 2021, doi: *10.1007/s11661-021-06192-x*.

[15] V. A. Esin *et al.*, "In Situ Synchrotron X-ray Diffraction and Dilatometric Study of Austenite Formation in a Multi-Component Steel: Influence of Initial Microstructure and Heating Rate," *Acta Materialia*, vol. 80, pp. 118–131, Nov. 2014, doi: *10.1016/j.actamat.2014.07.042*.

[16] A. Kamyabi-Gol, S. J. Clark, J. W. Gibbs, S. Sridhar, and P. F. Mendez, "Quantification of Evolution of Multiple Simultaneous Phase Transformations Using Dilation Curve Analysis (DCA)," *Acta Materialia*, vol. 102, pp. 231–240, Jan. 2016, doi: *10.1016/j.actamat.2015.09.025*.

[17] J. L. Loeppky, J. Sacks, and W. J. Welch, "Choosing the Sample Size of a Computer Experiment: A Practical Guide," *Technometrics*, vol. 51, no. 4, pp. 366–376, Nov. 2009, doi: *10.1198/TECH.2009.08040*.

Appendix A: Stochastic CA Module Code

```julia
module kmc_gpu

using AMDGPU
using ProgressMeter
using GLMakie
using Colors
using Printf
using Dates
using DelimitedFiles

const kB = 8.617333262f-5     # Blotzmann constant in eV/K
const R_gas = 8.314f0         # Universal gas constant (J/(mol*K))

struct SimParams
    inter_factor::Float32 # interphase surface energy orientation dependence factor (unitless)
    F_factor::Float32     # ferrite-ferrite interface orientation dependence factor (ev/um^2)
    A_factor::Float32     # austenite-austenite interface orientation dependece factor (ev/um^2)
    gamma_inter::Float32  # baseline interphase surface energy (ev/um^2)
    Q_mobility::Float32   # Activation energy for mobility factor (J/mol)
    M_0::Float32          # Pre-exponential factor for mobility (m^4/(J s))
    K_nuc::Float32        # Nucleation rate constant
    l::Float32            # Length of a single pixel in microns
    P_max::Float32        # limit on transition prob; this will force smaller timesteps
    θ_threshold::Float32
    M_GB_multiplier::Float32
end

function gibbs_ferrite(T::Float32)
    T_C = T - 273.0f0
    return -1.52f4 * T_C^2 - 3.33f7 * T_C - 4.26f9  # Gives eV/um^3
end

function gibbs_austenite(T::Float32)
    T_C = T - 273.0f0
    return -8.31f3 * T_C^2 - 4.83f7 * T_C + 3.12f9 # Gives eV/um^3
end

function abs_ΔG(T::Float32)
    return abs(gibbs_ferrite(T) - gibbs_austenite(T))
end

# Precalculated sin values for 0, 10, 20, 30, 40, 50, 60, 70, 80, 90 degrees
const SIN_LOOKUP = (0.0f0, 0.173648f0, 0.34202f0, 0.5f0, 0.642787f0, 0.766044f0,
```

```julia
    0.866025f0, 0.939692f0, 0.984807f0, 1.0f0)

function interface_energy(dθ::Int32, p1::Int32, p2::Int32, params::SimParams)
    # Get the index (e.g., dθ = 20 -> 20/10 + 1 = index 3)
    idx = (dθ ÷ Int32(10)) + Int32(1)

    sin_val = SIN_LOOKUP[idx]

    ifelse(p1 != p2,
        params.gamma_inter * (1.0f0 + params.inter_factor * sin_val),
        ifelse(p1 == 1,
            params.F_factor * sin_val,
            params.A_factor * sin_val
        )
    )
end

# Total energy for a single voxel (thermodynamic free energy)
function total_vox_energy(T::Float32, θ::Int32, p::Int32,
                          θ_n::NTuple{4, Int32}, p_n::NTuple{4, Int32}, params::SimParams)
    l = params.l
    l2 = l * l
    l3 = l2 * l

    # Bulk energy
    G_v = ifelse(p == 0, gibbs_austenite(T), gibbs_ferrite(T)) * l3

    sum_interface = 0.0f0

    sum_interface += interface_energy(abs(θ_n[1] - θ), p, p_n[1], params) * l2
    sum_interface += interface_energy(abs(θ_n[2] - θ), p, p_n[2], params) * l2
    sum_interface += interface_energy(abs(θ_n[3] - θ), p, p_n[3], params) * l2
    sum_interface += interface_energy(abs(θ_n[4] - θ), p, p_n[4], params) * l2

    return G_v + sum_interface
end

# Convert to a fixed-size Tuple of Tuples for the GPU
const POSSIBLE_STATES = Tuple((Int32(θ), Int32(p)) for θ in 0:10:90 for p in 0:1)

function get_M(T::Float32, params::SimParams)
    mobility = params.M_0 * exp(-params.Q_mobility / (R_gas * T))
    return mobility
end

# Growth kernel
function kmc_kernel!(S_new, S_old, R_matrix, Δt::Float32, T::Float32,
```

```
                                pass_parity::Int32, params::SimParams)
    i = workitemIdx().x + (workgroupIdx().x - 1) * workgroupDim().x
    j = workitemIdx().y + (workgroupIdx().y - 1) * workgroupDim().y

    N, M = size(S_old)

    if i <= N && j <= M
        # --- RED-BLACK CHECKERBOARD LOGIC ---
        if (i + j) % 2 != pass_parity
            S_new[i, j] = S_old[i, j]
            return nothing
        end

        θ, p = S_old[i, j]

        il = ifelse(i == 1, N, i - 1)
        ir = ifelse(i == N, 1, i + 1)
        jt = ifelse(j == 1, M, j - 1)
        jb = ifelse(j == M, 1, j + 1)

        θl, pl = S_old[il, j]
        θt, pt = S_old[i, jt]
        θr, pr = S_old[ir, j]
        θb, pb = S_old[i, jb]

        θ_n = (θl, θt, θr, θb)
        p_n = (pl, pt, pr, pb)

        current_energy = total_vox_energy(T, θ, p, θ_n, p_n, params)
        l = params.l

        candidates = ((θl, pl), (θt, pt), (θr, pr), (θb, pb))

        r = R_matrix[i, j]

        cum_P = 0.0f0
        chosen_state = (θ, p)

        for idx in 1:4
            cand_θ, cand_p = candidates[idx]

            is_current = (cand_θ == θ) && (cand_p == p)

            if !is_current
                E_cand = total_vox_energy(T, cand_θ, cand_p, θ_n, p_n, params)

```

```
                # ΔE is the change in energy. Negative means it is thermodynamically favorable.
                ΔE = E_cand - current_energy

                if ΔE < 0.0f0
                    driving_force = -ΔE
                    # Determine if the voxel (θ, p) being consumed sits on a grain boundary
                    # of its OWN current phase.
                    is_on_GB = false
                    θ_HA = params.θ_threshold # only high angle GBs are considered GBs

                    if (pl == p && abs(θ - θl) > Int32(θ_HA)) ||
                       (pt == p && abs(θ - θt) > Int32(θ_HA)) ||
                       (pr == p && abs(θ - θr) > Int32(θ_HA)) ||
                       (pb == p && abs(θ - θb) > Int32(θ_HA))
                        is_on_GB = true
                    end

                    mobility = get_M(T, params)
                    if is_on_GB
                        mobility *= params.M_GB_multiplier
                    end

                    v = mobility * driving_force * 0.160218f6
                    # 2. Macroscopic Capture Probability
                    P_capture = (v * Δt) / l

                    # Add this specific boundary's pull to the cumulative probability
                    cum_P += P_capture

                    if r <= cum_P
                        # sample from categorical distribution
                        chosen_state = candidates[idx]
                        break
                    end
                end
            end
        end

        S_new[i, j] = chosen_state
    end
    return nothing
end

function nucleate_kernel!(S, R_probs, R_seeds, Δt::Float32, T::Float32,
                          inv_kBT::Float32, params::SimParams)
```

```
    i = workitemIdx().x + (workgroupIdx().x - 1) * workgroupDim().x
    j = workitemIdx().y + (workgroupIdx().y - 1) * workgroupDim().y

    N, M = size(S)
    if i <= N && j <= M
        θ, p = S[i, j]

        # Get the current neighbors
        il = ifelse(i == 1, N, i - 1)
        ir = ifelse(i == N, 1, i + 1)
        jt = ifelse(j == 1, M, j - 1)
        jb = ifelse(j == M, 1, j + 1)

        θl, pl = S[il, j]
        θt, pt = S[i, jt]
        θr, pr = S[ir, j]
        θb, pb = S[i, jb]

        θ_n = (θl, θt, θr, θb)
        p_n = (pl, pt, pr, pb)

        # Propose a new state: flip the phase, and pick a random orientation
        new_p = ifelse(p == 0, Int32(1), Int32(0))
        # Ensure the random orientation maps to 0, 10, ..., 90 degree states
        new_θ = Int32((R_seeds[i, j] % 10) * 10)

        # Calculate the thermodynamic energy change
        E_old = total_vox_energy(T, θ, p, θ_n, p_n, params)
        E_new = total_vox_energy(T, new_θ, new_p, θ_n, p_n, params)
        ΔE = E_new - E_old

        metropolis_prob = ifelse(ΔE <= 0.0f0, 1.0f0, exp(-ΔE * inv_kBT))

        # K_nuc is an attempt frequency multiplier for spontaneous structural fluctuations
        freq = get_M(T, params) * abs(ΔE)/params.l  * 0.160218f6
        rate = params.K_nuc * freq * metropolis_prob
        P_trans = 1.0f0 - exp(-rate * Δt)

        if R_probs[i, j] < P_trans
            S[i, j] = (new_θ, new_p)
        end
    end
    return nothing
end

function find_max_velocity_kernel!(V_gpu, S_old, T::Float32, params::SimParams)
    i = workitemIdx().x + (workgroupIdx().x - 1) * workgroupDim().x
```

```
    j = workitemIdx().y + (workgroupIdx().y - 1) * workgroupDim().y

    N, M = size(S_old)

    if i <= N && j <= M
        θ, p = S_old[i, j]

        il = ifelse(i == 1, N, i - 1)
        ir = ifelse(i == N, 1, i + 1)
        jt = ifelse(j == 1, M, j - 1)
        jb = ifelse(j == M, 1, j + 1)

        θl, pl = S_old[il, j]
        θt, pt = S_old[i, jt]
        θr, pr = S_old[ir, j]
        θb, pb = S_old[i, jb]

        θ_n = (θl, θt, θr, θb)
        p_n = (pl, pt, pr, pb)

        current_energy = total_vox_energy(T, θ, p, θ_n, p_n, params)
        candidates = ((θl, pl), (θt, pt), (θr, pr), (θb, pb))

        local_max_v = 0.0f0

        # --- GB DETECTION ---
        is_on_GB = false
        if (pl == p && abs(θ - θl) > Int32(0)) ||
           (pt == p && abs(θ - θt) > Int32(0)) ||
           (pr == p && abs(θ - θr) > Int32(0)) ||
           (pb == p && abs(θ - θb) > Int32(0))
            is_on_GB = true
        end

        mobility = get_M(T, params)
        if is_on_GB
            mobility *= params.M_GB_multiplier
        end
        # -------------------------------

        for idx in 1:4
            cand_θ, cand_p = candidates[idx]

            if (cand_θ != θ) || (cand_p != p)
                E_cand = total_vox_energy(T, cand_θ, cand_p, θ_n, p_n, params)
                ΔE = E_cand - current_energy

```

```
                if ΔE < 0.0f0
                    # Use the multiplied mobility to find the true max velocity
                    v = mobility * abs(ΔE) * 0.160218f6
                    local_max_v = max(local_max_v, v)
                end
            end
        end

        # --- FLOAT ATOMIC TRICK ---
        # Only attempt a memory write if the pixel is actively moving.
        if local_max_v > 0.0f0
            # Reinterpret the Float32 bits as UInt32 to use hardware integer atomics
            val_uint = reinterpret(UInt32, local_max_v)
            # Need to do this because the ROCm atomic max api only works on integers
            AMDGPU.@atomic max(V_gpu[1], val_uint)
        end
    end
    return nothing
end

function track_state_over_time_gpu(S_init, t_array, T_func, params::SimParams ;
                                   Δt_floor = 1f-6,   #hard limit on smallest time step (s)
                                   plot_every = 10,
                                   record_video = true,
                                   framerate = 24,
                                   show_debug_stats=false,
                                   smart_steps=true,
                                   benchmark=false)

    l = params.l
    P_MAX = params.P_max

    if benchmark
        record_video = false
    end

    if ! benchmark
        # --- 1. Create Output Directory ---
        timestamp = Dates.format(Dates.now(), "yyyy-mm-dd_HHMMSS")
        out_dir = "sim_results_$timestamp"
        video_filename = "simulation_$timestamp.webm"
        mkpath(out_dir)
        println("Created output directory: $out_dir")

        # --- 2. Save Simulation Parameters to CSV ---
        params_file = joinpath(out_dir, "parameters.csv")
```

```
        open(params_file, "w") do io
            write(io, "Parameter,Value\n")
            for f in fieldnames(SimParams)
                val = getfield(params, f)
                write(io, "$f,$val\n")
            end
        end
    end

    N, M = size(S_init)
    n_steps = length(t_array) - 1

    S_init_32 = [Tuple{Int32, Int32}(s) for s in S_init]
    S_d1 = ROCArray(S_init_32)
    S_d2 = ROCArray(similar(S_init_32))

    vstream = nothing
    if record_video

        S_host = Array(S_d1)
        inst_T = Observable(T_func(t_array[1]) - 273.0f0)
        inst_time = Observable(Float32(t_array[1]))
        my_safe_t = Observable(Float32(t_array[2] - t_array[1]))
        # --- Visualization Setup ---
        f = Figure(size=(2560, 1440); fontsize=48)

        # Create observables for both Temperature and Time
        inst_T = Observable(T_func(t_array[1]) - 273.0f0)
        inst_time = Observable(Float32(t_array[1]))
        my_safe_t = Observable(Float32(t_array[2] - t_array[1]))

        img = Observable(tuple_to_rgb.(S_host))

    # Lift on BOTH observables to update the title dynamically
        dynamic_title = lift(inst_T, inst_time, my_safe_t) do myT, myt, myst
        # %8.4f means: take up 8 total characters, with exactly 4 decimal places
        @sprintf("Time: %8.4f s  |  Temp: %7.2f °C  |  Δt_safe: %7.4f", myt, myT, myst)
        end

        ax = Axis(f[1,1], title=dynamic_title, aspect=DataAspect())
            image!(ax, img, interpolate=false)
        # Initialize the Makie video stream attached to the figure
        vstream = VideoStream(f, framerate=framerate, compression=1)
    end

    # --- GPU Setup ---
    threads = (16, 16)
```

```
    blocks = (ceil(Int, N / threads[1]), ceil(Int, M / threads[2]))
    l2 = Float32(l * l)

    tracked_safe_times = Float32[]
    tracked_dt = Float32[]
    tracked_ferrite_frac = Float32[]
    tracked_phase_area = Float32[]
    tracked_diameter = Float32[]

    d_f_count = AMDGPU.zeros(Int32, N, M)
    d_pb_count = AMDGPU.zeros(Int32, N, M)
    d_gb_count = AMDGPU.zeros(Int32, N, M)

    R_nuc = AMDGPU.rand(Float32, N, M)
    R_seeds = AMDGPU.rand(UInt32, N, M)
    R_d_red = AMDGPU.rand(Float32, N, M)
    R_d_black = AMDGPU.rand(Float32, N, M)


    total_pixels = Float32(N * M)


    total_iters = 0

    # A single element to hold the max velocity bits
    max_v_gpu = AMDGPU.zeros(UInt32, 1)

    # --- Main Loop ---
    current_time = 0.0f0
    @showprogress for i in 1:n_steps
        Δt_macro = Float32(t_array[i + 1] - t_array[i])
        Δt_safe = Δt_macro  #initialize a value
        Δt_actual = Δt_safe #initialize a value
        current_T = Float32(T_func(current_time))
        inv_kBT = 1.0f0 / (kB * current_T)

        # Smart steps calculates the maximum velocity at every macro step
        # to adaptively select Δt for the subsequent microsteps.
        if smart_steps
            #Reset the single global tracker
            AMDGPU.fill!(max_v_gpu, UInt32(0))

            @roc groupsize=threads gridsize=blocks find_max_velocity_kernel!(max_v_gpu,
                                                                             S_d1,
                                                                             current_T,
                                                                             params)
            AMDGPU.synchronize()
```

```julia

            # Pull the UInt32 back to the CPU and translate it back to Float32
            max_v_uint = Array(max_v_gpu)[1]
            actual_max_v = reinterpret(Float32, max_v_uint)

            # Handle the edge case where nothing is moving (v = 0)
            if actual_max_v == 0.0f0
                # If nothing wants to move, take the largest allowed macro step
                Δt_safe = Δt_macro
            else
                # Calculate dynamic CFL limit based strictly on the fastest pixel at this moment
                Δt_safe = max(min(Δt_macro, (P_MAX * l) / actual_max_v), Δt_floor)
            end

            num_micro_steps = ceil(Int, Δt_macro / Δt_safe)
            Δt_actual = Δt_macro / Float32(num_micro_steps)
        else
            # If smart steps is off, calculate the maximum possible velocity
            # accross ANY transformation for every time step
            # Largest possible boundary energy
            gamma_max = interface_energy(Int32(90f0), Int32(0), Int32(1), params)
            max_boundary_release = 4.0f0 * gamma_max * l^2
            G_ferrite = gibbs_ferrite(current_T)
            G_austenite = gibbs_austenite(current_T)
            max_chemical_release = abs(G_ferrite - G_austenite) * (l^3)

            # Largest possible chemical driving force
            max_driving_force = max_boundary_release + max_chemical_release

            # Max velocity
            max_v = get_M(current_T, params) * max_driving_force * 0.160218f6

            # Courant–Friedrichs–Lewy (CFL) limit for the Cellular Automaton:
            # P_capture = (v * Δt) / l. We ensure 4 * P_capture never exceeds P_MAX.
            Δt_safe = max(min(Δt_macro, (P_MAX * l) / (4.0f0 * max_v)), Δt_floor)
            num_micro_steps = ceil(Int, Δt_macro / Δt_safe)

            Δt_actual = Δt_macro / Float32(num_micro_steps)
            @show Δt_actual
        end

        if Δt_safe < Δt_actual
            @warn "Step size may be too large; \n Δt_safe=$Δt_safe Δt_actual=$Δt_actual"
        end

```

```
        if show_debug_stats
            @show max_chemical_release
            @show max_boundary_release
            @show max_driving_force
            @show max_rate
            @show Δt_safe
            @show 1 - exp(-max_rate * Δt_actual)
        end

        # --- Calculate stats on the GPU every macro step ---
        @roc groupsize=threads gridsize=blocks compute_local_stats_kernel!(
            S_d1, d_f_count, d_pb_count, d_gb_count
        )
        AMDGPU.synchronize()

        # GPU performs the reduction
        total_ferrite = sum(d_f_count)
        total_pb = sum(d_pb_count)
        total_gb = sum(d_gb_count)

        # Final calc happens on CPU
        ferrite_fraction = Float32(total_ferrite) / total_pixels
        total_phase_area = Float32(total_pb) * l

        total_boundaries = Float32(total_pb + total_gb)
        if total_boundaries > 0.0f0
            P_L = total_boundaries / (2.0f0 * total_pixels * l)
            avg_diameter = 1.0f0 / P_L
        else
            avg_diameter = Float32(N * l)
        end

        push!(tracked_safe_times, Δt_safe)
        push!(tracked_dt, Δt_actual)
        push!(tracked_ferrite_frac, ferrite_fraction)
        push!(tracked_phase_area, total_phase_area)
        push!(tracked_diameter, avg_diameter)

        for _ in 1:num_micro_steps
            # --- 1. NUCLEATION PHASE ---
            AMDGPU.rand!(R_nuc)
            AMDGPU.rand!(R_seeds)
            T = Float32(T_func(current_time))

            @roc groupsize=threads gridsize=blocks nucleate_kernel!(
                S_d1, R_nuc, R_seeds, Δt_actual, T, inv_kBT, params
            )
```

```julia

            # --- 2. GROWTH PHASE (RED-BLACK CHECKERBOARD) ---
            # RED PASS (parity = 0)
            # Reads from S_d1, calculates updates for Red pixels, writes to S_d2
            # Black pixels are copied over identically
            AMDGPU.rand!(R_d_red)
            @roc groupsize=threads gridsize=blocks kmc_kernel!(
                S_d2, S_d1, R_d_red, Δt_actual, T, Int32(0), params
            )

            # BLACK PASS (parity = 1)
            # Reads from S_d2 (which now contains updated Reds), calculates updates
            # for Black pixels, writes back to S_d1
            AMDGPU.rand!(R_d_black)
            @roc groupsize=threads gridsize=blocks kmc_kernel!(
                S_d1, S_d2, R_d_black, Δt_actual, T, Int32(1), params
            )
            total_iters += 1
            current_time += Δt_actual
        end

        if record_video
            if i % plot_every == 0 || i == n_steps
                copyto!(S_host, S_d1)

                # Update visual observables
                inst_T[] = current_T - 273.0f0
                inst_time[] = Float32(t_array[i+1])
                img[] = tuple_to_rgb.(S_host)
                my_safe_t[] = Δt_safe

                # Prevents skipped frames
                yield()
                recordframe!(vstream)
            end
        end
    end

    @show total_iters
    if record_video
        full_vid_path = joinpath(out_dir, video_filename)
        println("Saving video to $video_filename...")
        save(full_vid_path, vstream)
        println("Video saved successfully!")
    end

    results = (
```

```
        times = tracked_safe_times,
        dt_sizes = tracked_dt,
        ferrite_fraction = tracked_ferrite_frac,
        phase_area = tracked_phase_area,
        avg_diameter = tracked_diameter
    )

    if ! benchmark
        # --- 3. Save Time-Series Results to CSV ---
        results_file = joinpath(out_dir, "timeseries_results.csv")
        open(results_file, "w") do io
            # Write the header row
            write(io, "Time_s,Temperature_C,Ferrite_Fraction,Phase_Area_um,Avg_Diameter_um,dt_safe,dt_actual\n")

            # Write the data rows
            for k in 1:n_steps
                t_val = t_array[k + 1]
                T_val = T_func(t_val) - 273.0f0 # Converted to Celsius for readability
                frac = tracked_ferrite_frac[k]
                p_area = tracked_phase_area[k]
                diam = tracked_diameter[k]
                dt_s = tracked_safe_times[k]
                dt_a = tracked_dt[k]

                write(io, "$t_val,$T_val,$frac,$p_area,$diam,$dt_s,$dt_a\n")
            end
        end
        println("Time-series results saved to $results_file")

        plot_simulation_results(results_file)
        plot_temperature_dependence(results_file)
    end
    return Array(S_d1), results
end

function tuple_to_rgb(state::Tuple{Int32, Int32})
    θ, p = state

    norm_θ = Float32(θ) / 90.0f0

    if p == 0
        return HSV(norm_θ * 60.0f0, 0.8f0, 0.9f0)
    else
        return HSV(200.0f0 + norm_θ * 60.0f0, 0.8f0, 0.9f0)
    end
end
```

```julia

# instantiate a frame with random orientation for every pixel
function instantiate_random_microstructure(N, M)
    [(rand(collect(0:10:90)), rand([0,0])) for i in 1:N, j in 1:M]
end

function tuples_to_tensor(A)
    permutedims(stack(A), (2, 3, 1))
end

# Constant cooling/heating thermal history
function generate_T_t(cr, T0, Δt, Nt)
    t = 0:Δt:(Δt*Nt)
    @show Δt*Nt
    T = T0 .- cr .* t
    @show T[end]
    return [t, T]
end


function generate_cc(T0, Tf, cr, Nt)
    Δt = (T0 - Tf)/cr / Nt
    t_f = Δt * Nt
    t = 0:Δt:t_f
    T = T0 .- cr .* t
    return collect(t), collect(T)
end

function cc_func(t; T0=(780f0+273f0), Tf=(500f0 + 273f0), cr = 1f0)
    return ifelse(T0 - cr * t > Tf, T0 - cr * t, Tf)
end


function thermal_cycle(T_max, T_min, Δt, period, N_cycles)
    t = collect(0:Δt:(N_cycles*period))

    N_steps = length(t)
    t_final = t[end]
    @show N_steps
    @show t_final
    max_rate = (T_max - T_min)/2 * 2π/period
    @show max_rate
    T = @. ((T_max - T_min) * cos(t/period * 2π) + (T_max + T_min))/2
    return t, T
end

```

```julia
function thermal_cycle_func(t; T_max = (780+273), T_min = 500 + 273, period = 900)::Float32
    return ((T_max - T_min) * cos(t/period * 2π) + (T_max + T_min))/2
end


# Generates a vornoi diagram type of microstructure with num_grains seeds
function initialize_real_microstructure(N::Int, M::Int, num_grains::Int; phase=1)
    # Initialize an empty host array
    S_init = Array{Tuple{Int32, Int32}}(undef, N, M)

    # 1. Generate random coordinates for the grain "seeds"
    seed_x = rand(1:N, num_grains)
    seed_y = rand(1:M, num_grains)

    # 2. Assign a random orientation (0, 10, ..., 90) and Phase 1 (Ferrite) to each seed
    seed_states = [(Int32(rand(0:10:90)), Int32(phase)) for _ in 1:num_grains]

    # 3. Populate the grid using the closest seed
    Threads.@threads for i in 1:N
        for j in 1:M
            min_dist_sq = Inf
            closest_seed = 1

            for k in 1:num_grains
                # Calculate distance in X and Y
                dx = abs(i - seed_x[k])
                dy = abs(j - seed_y[k])

                # Apply Periodic Boundary Conditions to the distance calculation
                # This ensures a grain on the right edge smoothly wraps to the left edge
                dx = min(dx, N - dx)
                dy = min(dy, M - dy)

                # Squared Euclidean distance (faster than taking the square root)
                dist_sq = dx^2 + dy^2

                if dist_sq < min_dist_sq
                    min_dist_sq = dist_sq
                    closest_seed = k
                end
            end

            # Assign the pixel the state of its closest seed
            S_init[i, j] = seed_states[closest_seed]
        end
    end
```

```julia

    return S_init
end

function compute_local_stats_kernel!(S, f_count, pb_count, gb_count)
    i = workitemIdx().x + (workgroupIdx().x - 1) * workgroupDim().x
    j = workitemIdx().y + (workgroupIdx().y - 1) * workgroupDim().y

    N, M = size(S)
    if i <= N && j <= M
        θ, p = S[i, j]

        # 1. Map Ferrite count (1 if Ferrite, 0 otherwise)
        f_count[i, j] = ifelse(p == 1, Int32(1), Int32(0))

        # 2. Map Boundary counts
        p_edges = Int32(0)
        g_edges = Int32(0)

        ir = ifelse(i == N, 1, i + 1)
        jb = ifelse(j == M, 1, j + 1)

        θr, pr = S[ir, j]
        θb, pb = S[i, jb]

        # Evaluate Right neighbor
        if p != -1 && pr != -1
            if p != pr
                p_edges += Int32(1)
            elseif θ != θr
                g_edges += Int32(1)
            end
        end

        # Evaluate Bottom neighbor
        if p != -1 && pb != -1
            if p != pb
                p_edges += Int32(1)
            elseif θ != θb
                g_edges += Int32(1)
            end
        end

        pb_count[i, j] = p_edges
        gb_count[i, j] = g_edges
    end
    return nothing
```

```
end

function plot_simulation_results(csv_path::String)
    # Read the data, separating the header row from the actual numbers
    data, header = readdlm(csv_path, ',', header=true)

    time_s = Float32.(data[:, 1])
    temp_C = Float32.(data[:, 2])
    ferrite_frac = Float32.(data[:, 3])
    avg_diameter = Float32.(data[:, 5])

    f = Figure(size = (1600, 600))

    # Axis 1: The JMAK S-Curve (Volume Fraction)
    ax1 = Axis(f[1, 1],
               #title = "Ferrite Volume Fraction vs. Time",
               xlabel = "Time (s)",
               ylabel = "Phase Fraction")

    ylims!(ax1, 0.0, 1.0)

    # Map the color to temp_C
    line1 = lines!(ax1, time_s, ferrite_frac,
                   color = temp_C, colormap = :inferno, linewidth = 3)

    # Axis 2: Average Grain Size Over Time
    ax2 = Axis(f[1, 2],
               #title = "Average Grain Diameter vs. Time",
               xlabel = "Time (s)",
               ylabel = "Diameter (µm)")

    lines!(ax2, time_s, avg_diameter,
           color = temp_C, colormap = :inferno, linewidth = 3)

    # --- Add the Colorbar ---
    Colorbar(f[1, 3], line1, label = "Temperature (°C)")

    #display(f)

    # 1. Extract the folder path from the CSV path
    out_dir = dirname(csv_path)

    # 2. Construct the full path for the new image
    img_path = joinpath(out_dir, "simulation_summary.png")

    # 3. Save the figure. px_per_unit = 2 doubles the pixel density for high res
    save(img_path, f, px_per_unit = 3)
```

```
    println("Plot successfully saved to: $img_path")

    return f
end

function plot_temperature_dependence(csv_path::String)
    # Read the data, separating the header row from the actual numbers
    data, header = readdlm(csv_path, ',', header=true)

    # Extract the columns based on the order we saved them
    time_s = Float32.(data[:, 1])
    temp_C = Float32.(data[:, 2])
    ferrite_frac = Float32.(data[:, 3])
    avg_diameter = Float32.(data[:, 5])

    # --- Visualization Setup ---
    # We use a taller figure to accommodate two distinct rows
    f = Figure(size = (1200, 1000))

    # Axis 1: Temperature vs. Time (Top row, spanning columns 1 and 2)
    ax1 = Axis(f[1, 1:2],
               #title = "Thermal Profile",
               xlabel = "Time (s)",
               ylabel = "Temperature (°C)")

    lines!(ax1, time_s, temp_C, color = :black, linewidth = 3)

    # Axis 2: Phase Evolution vs. Temperature (Bottom Left)
    ax2 = Axis(f[2, 1],
               #title = "Phase Evolution vs. Temperature",
               xlabel = "Temperature (°C)",
               ylabel = "Ferrite Area Fraction")

    ylims!(ax2, 0.0, 1.0)
    # Mapping color to time_s so you can see the chronological path of the curve
    lines!(ax2, temp_C, ferrite_frac,
           color = time_s, colormap = :viridis, linewidth = 3)

    # Axis 3: Grain Size vs. Temperature (Bottom Right)
    ax3 = Axis(f[2, 2],
               #title = "Grain Growth vs. Temperature",
               xlabel = "Temperature (°C)",
               ylabel = "Average Grain Diameter (μm)")

    lines!(ax3, temp_C, avg_diameter,
           color = time_s, colormap = :viridis, linewidth = 3)

```

```
    # Add a colorbar for time on the far right of the bottom row
    Colorbar(f[2, 3], colormap = :viridis, limits = extrema(time_s), label = "Time (s)")

    #display(f)

    out_dir = dirname(csv_path)
    img_path = joinpath(out_dir, "temperature_dependence_summary.png")

    save(img_path, f, px_per_unit = 3)
    println("Plot successfully saved to: $img_path")

    return f
end
end
```